

Práctica 3: Listas Doblemente Enlazadas

Navegación bidireccional

AED I — Bloque I

2 h en clase + ~1 h en casa · Python

Algoritmia y Estructuras de Datos | Ingeniería Informática – Inteligencia Artificial

¿Para qué sirve esto?

¿Quién ha usado CTRL+Z alguna vez?

- Historial de Navegador (esto lo vamos a revisar hoy)
- Spotify
- OS caché
- LinkedHashMap

¿Qué vamos a hacer?

1. Repasar la lista simple y damos el salto a la lista doble
2. Programamos juntos, en directo, una `ListaDoble` completa
3. Resolvemos el reto del palíndromo con dos punteros
4. Vemos la arquitectura de un historial de navegador con atrás/adelante.
5. Cerramos con un test individual sobre lo trabajado hoy

Objetivos de la práctica

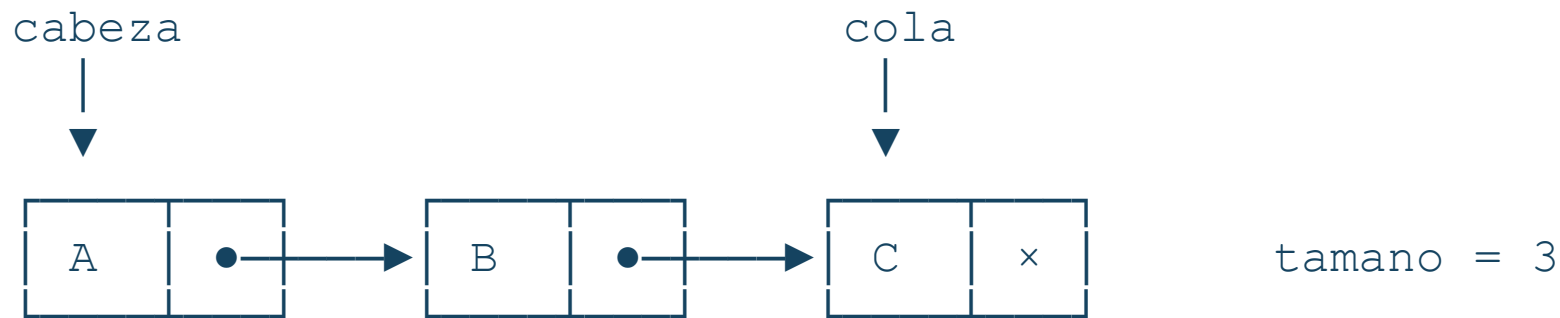
1. Entender por qué un segundo puntero cambia la complejidad de lo que una lista enlazada puede hacer.
2. Implementar desde cero una lista doblemente enlazada en Python, comprobada con su invariante y tests.
3. Aplicar la navegación bidireccional para resolver problemas clásicos (p. ej.: palíndromo).
4. Extender una lista doblemente enlazada para resolver el historial de navegación web.
5. Hacer que la lista se use como cualquier secuencia de Python: `len(l)`, `for x in l`, `reversed(l)`.

Flujo de la sesión

Fase	Duración	Actividad
Motivación y repaso	20 min	Ejemplos reales; de lista simple a doble
Parte 1: Programar la lista doble	45 min	Live coding en Python + tests
Parte 2: ¿Es palíndromo?	15 min	Implementar, probar y comparar
Parte 3: Historial de navegador	15 min	Arquitectura y traza (se programa en casa)
Cierre y test individual	15 min	Qué queda para casa + preguntas

Repaso: La lista simplemente enlazada

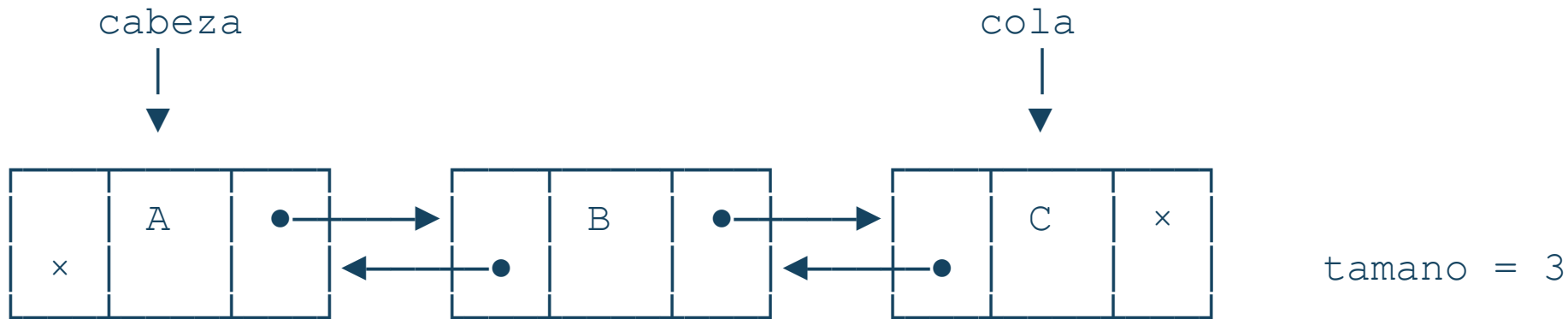
En una lista simplemente enlazada cada nodo guarda un **dato** y un único puntero al **siguiente**. Solo se puede recorrer en un sentido, desde `cabeza` hasta `None`.



- Eliminar el **último** nodo: hay que recorrer toda la lista para encontrar el penúltimo y hacer que su `siguiente` sea `None`. **$O(n)$** .
- Insertar **antes** de un nodo dado: mismo problema, hay que buscar el anterior desde la cabeza. **$O(n)$** .

De simple a doblemente enlazada

Misma lista, con un solo puntero más en cada nodo: `anterior`.



Cada `NodoDoble`: `anterior` | `dato` | `siguiente`. **La** `ListaDoble`: `cabeza`, `cola` y `tamano`.

`x = None`: `cabeza.anterior` is `None` y `cola.siguiente` is `None`.

Eliminar el último nodo

¿Cuánto cuesta? **O(1)**: `self.cola.anterior` ya es el penúltimo, no hay que recorrer nada.

```
nodo = self.cola                # el que se va
self.cola = nodo.anterior       # 1. cola retrocede
self.cola.siguiente = None      # 2. el nuevo último ya no apunta a nodo
nodo.anterior = None            # 3. nodo queda desenganchado
self.tamano -= 1
```

- Si solo había un elemento: `self.cabeza = self.cola = None`.
- En la plantilla: `delete_last()` es `delete_node(self.cola)`, y en una lista vacía lanza `IndexError` (como `list.pop()`).

Insertar X al inicio

cabeza -> [A <-> B <-> C] <- cola

```
nuevo = NodoDoble(dato)
if self.cabeza is None:           # lista vacia: EL caso especial
    self.cabeza = self.colas = nuevo
else:
    nuevo.siguiente = self.cabeza # 1. X.siguiente = cabeza
    # 2. X.anterior = None (ya lo es al crear el nodo)
    self.cabeza.anterior = nuevo  # 3. cabeza.anterior = X
    self.cabeza = nuevo           # 4. cabeza = X
self.tamano += 1
```

cabeza -> [**X** <-> A <-> B <-> C] <- cola

Caso especial, lista vacía: `self.cabeza = self.colas = nuevo`

Eliminar un nodo en general

$\text{cabeza} \rightarrow [A \leftrightarrow B \leftrightarrow C] \leftarrow \text{cola}$

Si ya tenemos el nodo B, lo quitamos en **O(1)** sin buscar a su anterior:

```
if nodo.anterior is not None:           # A.siguiente = C
    nodo.anterior.siguiente = nodo.siguiente
else:                                   # B era la cabeza
    self.cabeza = nodo.siguiente
if nodo.siguiente is not None:          # C.anterior = A
    nodo.siguiente.anterior = nodo.anterior
else:                                   # B era la cola
    self.col = nodo.anterior
nodo.anterior = nodo.siguiente = None   # B desenganchado
self.tamano -= 1
return nodo.dato
```

Único elemento: entran los dos `else` $\rightarrow \text{cabeza} = \text{cola} = \text{None}$.

$\text{cabeza} \rightarrow [A \leftrightarrow C] \leftarrow \text{cola}$

El invariante de la lista doble

Lo que SIEMPRE debe ser cierto al terminar cualquier operación:

- `tamano == 0` $\Leftrightarrow$ `cabeza is None` $\Leftrightarrow$ `cola is None`
- `cabeza.anterior is None` y `cola.siguiente is None`
- de cabeza a cola por `.siguiente` hay exactamente `tamano` nodos
- `x.anterior.siguiente is x` para todo nodo salvo la cabeza

`invariante_correcto()` ya está en la plantilla: los tests lo comprueban después de **cada** operación.

«invariante roto tras `delete_last`» = el resultado parece bueno, pero algún puntero ha quedado mal por dentro.

Cómo trabajamos: plantilla y tests

Completa lista_doble.py en orden y ejecuta el grupo de tests de cada paso:

```
cd python
python3 -m unittest -v test_lista_doble.PruebasInsercion

PruebasInsercion          insert_last, __reversed__
PruebasInsercionRelativa  insert_before, insert_after
PruebasBorrado            delete_node, delete_first, delete_last
PruebaContraList          2000 operaciones contra una list (oráculo)
PruebasPalindromo         es_palindromo
```

Python por debajo: `len(l) → __len__ · for x in l → __iter__ · reversed(l) → __reversed__`

`None ≈ nullptr · compara nodos con is · borrar en lista vacía → IndexError (como list.pop())`

Parte 1: Implementando la lista doble (1)

¡Abrimos nuestro editor!

- Clase Nodo con 2 punteros:

```
class NodoDoble:
    # __slots__: sin diccionario de atributos -> menos memoria
    __slots__ = ("dato", "siguiente", "anterior")

    def __init__(self, dato):
        self.dato = dato
        self.siguiente = None
        self.anterior = None
```

Parte 1: Implementando la lista doble (2)

Clase ListaDoble (en la plantilla lista_doble.py):

```
class ListaDoble:
    def __init__(self, iterable=None):      # hecho: cabeza, cola, tamaño
    def __len__(self):                      # hecho: len(l)
    def __iter__(self):                     # hecho: for x in l
    def __reversed__(self):                 # TODO: reversed(l)
    def insert_first(self, dato):           # hecho (ejemplo: 4 punteros)
    def insert_last(self, dato):            # TODO
    def print_forward(self):                # hecho (usa for)
    def print_backward(self):               # hecho (usa reversed)
```

cabeza -> [A <-> B <-> C] <- cola

Parte 1: Implementando la lista doble (3)

- Inserción general:

```
def insert_before(self, nodo, dato):    # TODO
def insert_after(self, nodo, dato):    # TODO
```

Casos especiales: si `nodo` es la cabeza, `insert_first`; si es la cola, `insert_last`.

Todos los `insert_*` devuelven el nodo nuevo, y `l.buscar(dato)` da el nodo de un dato.

`cabeza -> [A <-> B <-> C] <- cola`

Parte 1: Implementando la lista doble (4)

- Borrando nodos:

```
def delete_node(self, nodo):    # TODO: devuelve nodo.dato
def delete_first(self):        # TODO: delete_node(self.cabeza)
def delete_last(self):         # TODO: delete_node(self cola)
```

Lista vacía: `raise IndexError` (como `list.pop()`). Y `delete_node` deja `nodo.anterior` y `nodo.siguiente` a `None`.

`cabeza -> [A <-> B <-> C] <- cola`

Parte 2: ¿Es palíndromo? (1)

cabeza -> [1 <-> 2 <-> 3 <-> 2 <-> 1] <- cola



cabeza -> [1 <-> 2 <-> 3 <-> 4 <-> 5] <- cola



```
def es_palindromo(self):
```

- Recorremos desde cabeza y cola a la vez. ¿Basta con tamaño/2?

Parte 2: ¿Es palíndromo? (2)

```
def es_palindromo(self):  
    if self.tamano <= 1:  
        return True  
  
    inicio = self.cabeza  
    fin = self.cola  
  
    for _ in range(self.tamano // 2):  
        if inicio.dato != fin.dato:  
            return False  
        inicio = inicio.siguiente  
        fin = fin.anterior  
  
    return True
```

Ampliación (opcional): otras operaciones

`intercalar(otra)` — mezcla alternando; otra no cambia

```
[1 <-> 3 <-> 5].intercalar([2 <-> 4 <-> 6])  
-> [1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6]
```

`reverse()` — sin crear nodos: intercambia anterior y siguiente

```
[1 <-> 2 <-> 3].reverse() -> [3 <-> 2 <-> 1]
```

`rotar(k, derecha=True)` — mueve el corte, sin crear nodos

```
[1 <-> 2 <-> 3 <-> 4 <-> 5].rotar(2) -> [4 <-> 5 <-> 1 <-> 2 <-> 3]
```

Parte 3: Historial de navegador (1)

Historial: leetcode.com <-> google.com <-> facebook.com <-> youtube.com

 $\wedge$

actual (estás en facebook.com)

visit("linkedin.com") desde aquí:

1. youtube.com se descarta (estaba "por delante")
2. se enlaza un nuevo nodo linkedin.com justo después de facebook.com
3. actual pasa a apuntar a linkedin.com

Historial: leetcode.com <-> google.com <-> facebook.com <-> linkedin.com

 $\wedge$

actual

Parte 3: Historial de navegador (2)

- No necesitamos ninguna estructura nueva: la misma ListaDoble y un único puntero, actual, al NODO en el que estamos.
- Tres operaciones: visit, back y forward. Se programan en casa con los tests de test_historial.py.

```
class BrowserHistory:
    def __init__(self, homepage):          # hecho
        self.paginas = ListaDoble()
        self.actual = self.paginas.insert_last(homepage)

    def visit(self, url):                  # TODO: descartar lo de delante, enlazar, mover actual
    def back(self, steps):                  # TODO: devuelve self.actual.dato
    def forward(self, steps):               # TODO: devuelve self.actual.dato
```

Parte 3: Historial de navegador (3)

Traza de navegación:

```
BrowserHistory("leetcode.com")  
visit("google.com")
```

```
visit("facebook.com")  
visit("youtube.com")
```

```
back(1)    -> "facebook.com"  
back(1)    -> "google.com"  
forward(1) -> "facebook.com"  
visit("linkedin.com")
```

```
forward(2) -> "linkedin.com"  
back(2)    -> "google.com"  
back(100)  -> "leetcode.com"
```

```
actual = leetcode.com  
leetcode.com <-> google.com  
actual = google.com  
... <-> google.com <-> facebook.com  
... <-> facebook.com <-> youtube.com  
actual = youtube.com  
actual retrocede 1 posición  
actual retrocede 1 posición más  
actual avanza 1 posición  
se descarta youtube.com; nuevo nodo tras  
facebook.com  
no hay más "adelante" que linkedin.com  
actual retrocede una posición  
tope en el primer nodo, no vuelve más
```

Cierre y actividad para casa

Obligatorio en casa (~1 h)

- `BrowserHistory`: visit, back, forward → `test_historial.py`
- Juez, 2 problemas: `lista_doble.py` y `envio_historial.py` (lo crea `generar_envio.py`)
- `RESPUESTAS.md`: tabla de costes y 3 preguntas

Ampliación opcional (no puntúa)

- `reverse()`, `intercalar()`, `rotar()` · `comparativa.py` (medir con el reloj) · versión en C++

Ahora: test individual

¡Manos al código!

Siguiente práctica: Pilas